

SE3090 – Software Engineering Frameworks

Lecture 03

C#, .NET Fundamentals, ASP.NET Core and REST API Development

Stack: C# 12 · .NET 8 · ASP.NET Core Duration: 3 hours Learning outcomes: LO2 · LO3 · LO4

Year 3 • Semester 1 • 4 Credits

Lecturer: Eishan Weerasinghe • eishan.w@slit.lk

Where This Lecture Fits in SE3090

L01

Framework
fundamentals & trends

L02

Advanced React
frontend

TODAY

C#, .NET, ASP.NET Core
& REST APIs

Next

Database, AuthN/AuthZ
& integration

- **Module content areas 2(c)–2(e): REST communication, framework-based backend development, server-side applications**
- The API tier you used as a 'black box' in the database lecture becomes today's subject
- Directly assessed in labs, the mini hackathon and the Main Assignment backend (25%)
- Foundation for CI/CD, deployment and AI-assisted backend development in later lectures

Lecture Learning Outcomes

LO2

Apply suitable frameworks & tools

Build a working ASP.NET Core REST API using C# OOP, controllers, services, models and routing.

LO3

Use best practices for integration & quality

Structure a maintainable layered backend with DI, validation, exception handling, logging and tested endpoints (Swagger/Postman).

LO4

Select & justify approaches

Justify the choice of .NET/ASP.NET Core and REST design decisions (resources, methods, status codes) for project requirements.

Why C#, .NET and ASP.NET Core Matter



Industry footprint

Enterprise systems, banking, government, healthcare, games (Unity) and cloud services run on .NET — strong demand in Sri Lanka and globally.



Performance

ASP.NET Core consistently ranks among the fastest mainstream web frameworks (TechEmpower benchmarks) — millions of requests/sec on Kestrel.



One language, many targets

C# covers web APIs, desktop, mobile (MAUI), games, IoT and cloud functions — skills transfer across domains.



Modern & open source

Cross-platform (Windows/Linux/macOS), MIT-licensed, 6-monthly releases, strong typing + rich tooling (Visual Studio, Rider, VS Code).

Lecture Roadmap – 3 Hours

1

C# fundamentals

Syntax · types · control flow · methods · classes

~30 min

2

OOP in C#

Encapsulation · inheritance · polymorphism · abstraction · interfaces

~35 min

3

.NET & ASP.NET Core

Platform · runtime · DI · middleware pipeline

~25 min

4

REST API architecture

Resources · HTTP methods · status codes · JSON · lifecycle

~35 min

5

Controllers, services, models & routing

MVC-style structure · DTOs · attribute routing

~25 min

6

Requests, responses & testing + best practices

Binding · responses · Swagger · Postman · layered design

~30 min

PART 1

C# Programming Fundamentals

- Role of C# in modern software development
- Program structure, variables, types and operators
- Control structures and methods
- First look at classes and objects

The Role of C# in Modern Development

- **General-purpose, statically typed, object-oriented language (2000, Microsoft; now open standard)**
 - Compiled to IL (Intermediate Language), executed by the .NET runtime (JIT)
- **Static typing = errors caught at compile time, great IDE support, safer refactoring**
- **Modern features: async/await, LINQ, pattern matching, records, nullable reference types**
- **Where you'll meet it:**
 - Web APIs & microservices (ASP.NET Core) — our focus
 - Enterprise/desktop apps, Azure cloud functions, Unity games, MAUI mobile

C# vs languages you know

vs JavaScript:

compile-time types instead of runtime surprises; classes are real, not prototypes

vs Java:

very similar feel; C# adds properties, LINQ, async/await ergonomics, value types

vs Python:

more ceremony, but far faster and safer for large team codebases

Anatomy of a C# Program

Program.cs

```
// Program.cs — modern top-level statements
using System;                                // import namespace

Console.WriteLine("Hello, SE3090!");

// Classic explicit form (same program)
namespace LankaMart.Api;                    // organises code

public class Program {
    public static void Main(string[] args) {
        Console.WriteLine("Hello, SE3090!");
    }
}
```

Building blocks

- using — imports a namespace
- namespace — logical grouping; mirrors folder structure
- class — the unit of code; (almost) everything lives in one
- Main / top-level statements — the entry point
- Statements end with ; — blocks use { }

Try it: dotnet new console → dotnet run. ASP.NET Core's Program.cs uses exactly this top-level style.

Variables, Data Types and Operators

```
int    qty      = 3;           // whole numbers
double rating   = 4.7;        // floating point
decimal price   = 2500.00m;    // money — exact!
bool    inStock = true;
string name     = "Wireless Mouse";
char    grade   = 'A';

var total = price * qty;       // type inferred: decimal
string? note = null;          // nullable reference

string label =
    $"{name}: {total:C} ({(inStock ? "in" : "out")}");
```

Key points

- **decimal for money — never double (rounding errors)**
- var = inference, still strongly typed
- string? — nullable reference types make null explicit
- \$"..." string interpolation
- Operators as expected: + - * / %, == != < >, && || !

Backend link: these types map directly to your PostgreSQL columns — decimal ↔ NUMERIC, int ↔ INT, string ↔ TEXT, bool ↔ BOOLEAN. Choosing the C# type IS a schema decision.

Control Structures

```
// if / else
if (qty <= 0)
    return "Quantity must be positive";
else if (qty > stock)
    return "Not enough stock";

// switch expression (modern C#)
var fee = method switch {
    "card"    => 0m,
    "cod"     => 250m,
    _         => throw new ArgumentException()
};
```

```
// loops
for (int i = 0; i < items.Count; i++)
    Console.WriteLine(items[i]);

foreach (var item in items) // preferred
    total += item.Price * item.Qty;

while (retries < 3 && !ok)
    ok = TryConnect(ref retries);

// LINQ – declarative alternative
var total = items.Sum(i => i.Price * i.Qty);
```

Guard clauses (validate-and-return-early) keep backend methods flat and readable — you'll see them in every controller and service today.

Methods and Parameters

```
// access return-type Name(parameters)
public decimal CalculateTotal(
    List<OrderItem> items,
    decimal discountPct = 0)          // optional param
{
    if (items is null || items.Count == 0)
        throw new ArgumentException("Empty order");

    var subtotal = items.Sum(i => i.Price * i.Qty);
    return subtotal * (1 - discountPct / 100m);
}

// expression-bodied – for one-liners
public bool IsInStock(Product p) => p.StockQty > 0;
```

Reading the signature

- public — visible outside the class
- decimal — return type (void = none)
- Parameters are typed; optional ones take defaults
- PascalCase method names, verbs first (CalculateTotal, GetById)
- One method = one job — small methods are testable methods

Classes and Objects — First Look

```
public class Product
{
    // properties – data with controlled access
    public long    Id        { get; set; }
    public string  Name      { get; set; } = "";
    public decimal Price     { get; set; }
    public int     StockQty  { get; set; }

    // behaviour – methods that use the data
    public bool CanFulfil(int qty) => StockQty >= qty;
}

var mouse = new Product { Name = "Mouse", Price = 2500m };
bool ok = mouse.CanFulfil(2);    // object in action
```

Class vs object

- Class = blueprint (the Product design)
- Object = a concrete instance made with new
- Properties hold state; methods provide behaviour
- new creates an instance; object initializer { } sets properties
- This Product maps 1:1 to the products table from Lecture 03

Checkpoint 01 – Quick Questions



Answer with your neighbour — 4 minutes

Q1

Why must an order total use decimal rather than double? What could go wrong in production?

Q2

`var x = 10; x = "ten";` — does this compile? What does that tell you about `var`?

Q3

Rewrite mentally with a guard clause: `if (user != null) { if (user.IsActive) { /* 30 lines */ } }`

Q4

In one sentence: what is the difference between the `Product` class and the `mouse` object?

PART 2

Object-Oriented Programming in C#

- Constructors and properties
- Encapsulation · inheritance · polymorphism · abstraction
- Interfaces — the backbone of ASP.NET Core design
- Practical OOP for backend models and services

Constructors and Properties

```
public class Order
{
    public long Id { get; private set; }      // read-only outside
    public long CustomerId { get; }           // immutable
    public string Status { get; private set; }
    public DateTime CreatedAt { get; } = DateTime.UtcNow;

    // constructor — guarantees a valid object
    public Order(long customerId)
    {
        if (customerId <= 0)
            throw new ArgumentException("Invalid customer");
        CustomerId = customerId;
        Status = "PENDING";
    }
}
```

Why it matters

- Constructor = the object's front door: no Order exists without a valid customer
- Properties control WHO can change WHAT (get; private set;)
- An invalid state becomes unrepresentable — fewer bugs by design
- UtcNow default: timestamps in UTC, echoing Lecture 03

Encapsulation — Protecting Object State

```
public class BankAccount
{
    private decimal _balance;           // hidden state

    public decimal Balance => _balance; // read-only view

    public void Deposit(decimal amount)
    {
        if (amount <= 0) throw new ArgumentException();
        _balance += amount;
    }

    public bool Withdraw(decimal amount) =>
        amount > 0 && amount <= _balance
        ? (_balance -= amount) >= 0 : false;
}
```

The principle

- State is private; behaviour is public
- All changes pass through methods that enforce the rules
- Impossible to set `_balance = -1,000,000` from outside
- Access modifiers: `public` · `private` · `protected` · `internal`
- Rule of thumb: start private, open up only with reason

Backend reality: encapsulation is why business rules live in services/domain objects — not spread across controllers that poke at raw data.

Inheritance — Sharing Structure and Behaviour

```
public class User                                // base class
{
    public long Id { get; set; }
    public string Name { get; set; } = "";
    public string Email { get; set; } = "";
    public virtual string Describe() => $"User {Name}";
}

public class Staff : User                        // derived class
{
    public string Branch { get; set; } = "";
    public override string Describe()
        => $"Staff {Name} ({Branch})";
}
```

Mechanics

- : User — Staff IS-A User; inherits all members
- virtual — base allows overriding
- override — derived replaces behaviour
- base.Member reaches the parent
- C# = single inheritance (one base class only)

Use with care: prefer inheritance for true IS-A relationships (Staff is a User). For sharing capabilities, prefer composition or interfaces — "composition over inheritance".

Polymorphism — One Interface, Many Behaviours

```
List<User> users = new()
{
    new User { Name = "Amal" },
    new Staff { Name = "Nadia", Branch = "Kandy" },
    new Admin { Name = "Ruwan" }
};

foreach (var u in users)
    Console.WriteLine(u.Describe());

// Output - same call, three behaviours:
// User Amal
// Staff Nadia (Kandy)
// Admin Ruwan [full access]
```

What just happened

- The variable's type is User; the OBJECT decides which Describe runs (runtime dispatch)
- Calling code stays generic — new user types need zero changes here
- This is the Open/Closed Principle: open for extension, closed for modification
- Same idea powers ASP.NET Core: the framework calls YOUR controller methods polymorphically

Abstraction and Interfaces

```
// interface – a pure contract, no implementation
public interface IEmailSender
{
    Task SendAsync(string to, string subject, string body);
}

public class SmtplibEmailSender : IEmailSender { ... }
public class FakeEmailSender : IEmailSender { ... } // tests

// consumers depend on the CONTRACT, not a class:
public class OrderService(IEmailSender email) { ... }
```

Interface vs abstract class

- Interface: contract only; a class can implement MANY
- Abstract class: partial implementation + contract; single inheritance
- Naming: IName (IEmailSender, IProductService)

Why backends are built on interfaces

Swap implementations (SMTP → SendGrid) without touching consumers · plug in fakes for unit tests · ASP.NET Core dependency injection registers and resolves BY INTERFACE — this single concept unlocks the next part of the lecture.

OOP in Practice — A Backend Domain Model

```
public class Order
{
    private readonly List<OrderItem> _items = new();
    public IReadOnlyList<OrderItem> Items => _items;
    public string Status { get; private set; } = "PENDING";
    public decimal Total => _items.Sum(i => i.LineTotal);

    public void AddItem(Product p, int qty)
    {
        if (Status != "PENDING")
            throw new InvalidOperationException("Order locked");
        if (!p.CanFulfil(qty))
            throw new InvalidOperationException("No stock");
        _items.Add(new OrderItem(p.Id, qty, p.Price));
    }

    public void Confirm() => Status = "CONFIRMED";
}
```

Spot all four pillars

- Encapsulation: `_items` private, rules in `AddItem`
- Abstraction: callers see `AddItem/Confirm`, not list mechanics
- Computed property: `Total` derives from items — never stored stale
- Snapshot price captured (`p.Price`) — Lecture 03's `unit_price` rule, now in code!
- Polymorphism & inheritance arrive via EF Core & controllers

Checkpoint 02 – OOP Quick Questions



Pairs — 4 minutes, then we discuss

Q1

User `u = new Admin(); u.Describe();` — whose `Describe` runs, and what is this mechanism called?

Q2

A teammate makes `Order.Status` a public `{ get; set; }`. What bug becomes possible that was impossible before?

Q3

`PaymentService` needs card, cash and bank-transfer handling that can grow. Interface or abstract class for the payment types — and why?

Q4

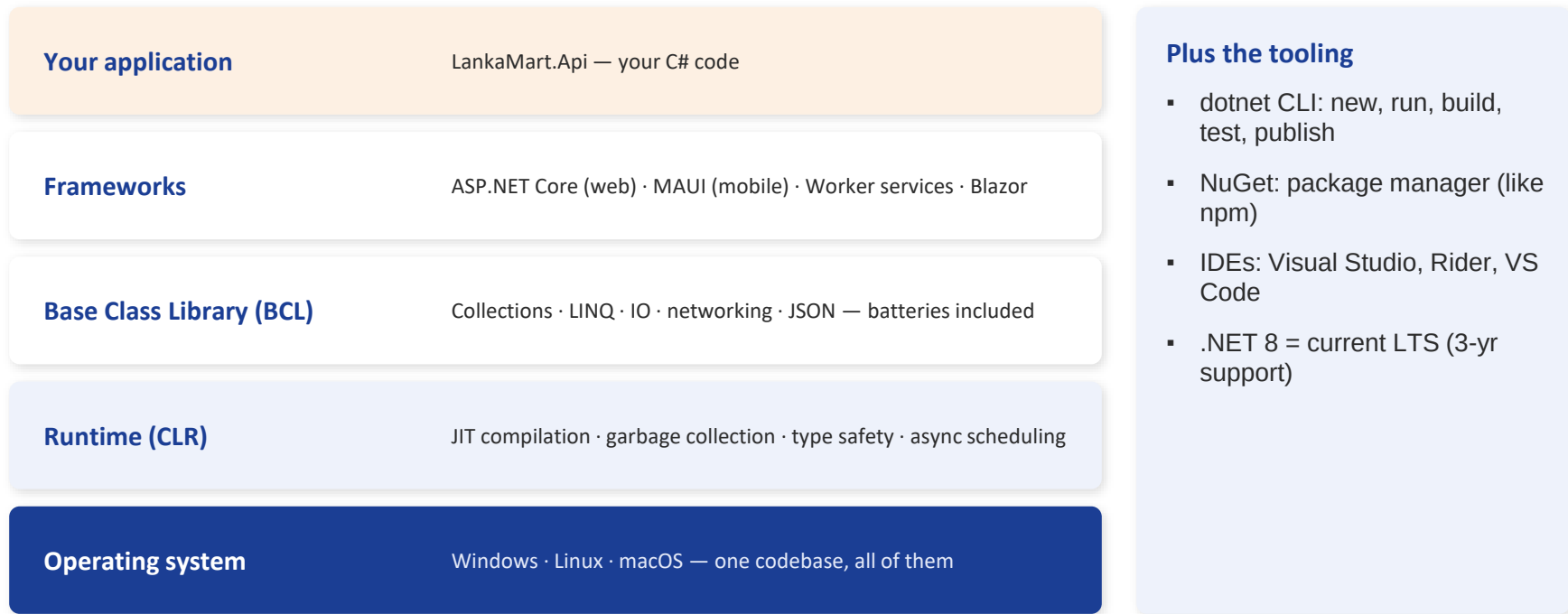
True or false: 'Inheritance is the main tool for code reuse in modern backends.' Defend your answer.

PART 3

.NET and ASP.NET Core

- What .NET is: platform, runtime, libraries, CLI
- .NET vs C# vs ASP.NET Core — untangled
- Why ASP.NET Core for modern web APIs
- Dependency injection & the middleware pipeline

What is .NET? Platform, Runtime and Ecosystem



A development platform: a runtime that executes your code + a huge standard library + tooling to build and run it.

.NET vs C# vs ASP.NET Core — Untangled



C#

The language

- Syntax, types, OOP — what you WRITE
- Compiles to IL
- Other .NET languages exist (F#, VB)

The language you speak



.NET

The platform

- Runtime (CLR) + libraries (BCL) + tools
- WHERE your code runs
- Cross-platform, open source

The country and its roads



ASP.NET Core

The web framework

- Built ON .NET for web apps & APIs
- Routing, controllers, DI, middleware, Kestrel
- What we use to SERVE HTTP

A delivery company using those roads

Analogy: C# is to .NET what JavaScript is to Node.js; ASP.NET Core is the Express/NestJS of that world — language → platform → web framework.

- You write C#, which runs on .NET, using ASP.NET Core to handle the web part

Why ASP.NET Core for Modern Web APIs



Fast by default

Kestrel + async I/O top the TechEmpower benchmarks among mainstream frameworks; async/await is idiomatic, not bolted on.



Cross-platform & container-ready

Develop on Windows/macOS, deploy to Linux Docker containers — the standard cloud path (Azure, AWS).



Batteries included

Routing, model binding, validation, DI container, configuration, logging, auth, OpenAPI — in the framework, not 30 npm packages.



Enterprise-grade & supported

LTS releases, security patches, massive ecosystem, first-class tooling — safe choice to justify for serious systems (LO4).

Dependency Injection (DI) — The Framework's Backbone

```
// 1. Register (Program.cs) - interface → implementation
builder.Services.AddScoped<IProductService, ProductService>();
builder.Services.AddScoped<IEmailSender, SmtplibEmailSender>();
builder.Services.AddDbContext<AppDbContext>(...);

// 2. Request (constructor) - the container injects
public class ProductsController(IProductService svc)
    : ControllerBase
{
    [HttpGet] public Task<List<ProductDto>> GetAll()
        => svc.GetAllAsync();    // svc arrived automatically
}
```

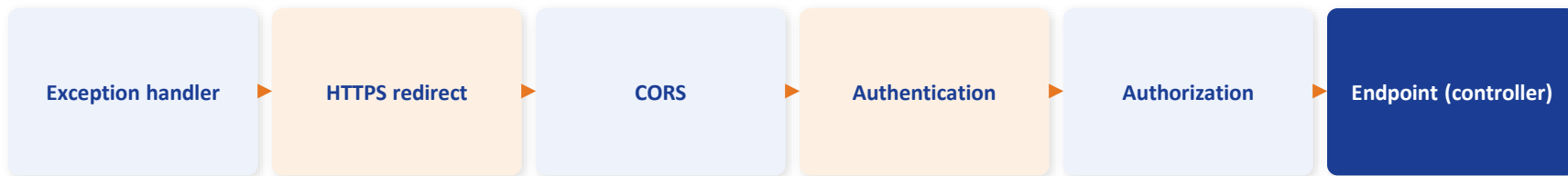
Lifetimes

- Singleton — one instance forever (config, caches)
- **Scoped** — one per HTTP request (services, DbContext)
- Transient — new every time (lightweight, stateless)

Why: classes declare WHAT they need (interfaces); the container decides WHICH implementation and WHEN to create it. Result: loose coupling, easy testing (inject fakes), and one place to change wiring. This is 'Inversion of Control' — you don't new your dependencies; they are handed to you.

The Middleware Pipeline

Request →



← Response (back through the same chain)

```
// Program.cs — order matters!
app.UseExceptionHandler("/error");
app.UseHttpsRedirection();
app.UseCors("Frontend");
app.UseAuthentication(); // who are you?
app.UseAuthorization(); // may you?
app.MapControllers();
```

Mental model

- Each middleware can inspect, modify, short-circuit, or pass along
- 401 from JWT middleware = a short-circuit — your controller never runs
- Order matters: AuthN must precede AuthZ; exception handler wraps everything (first)

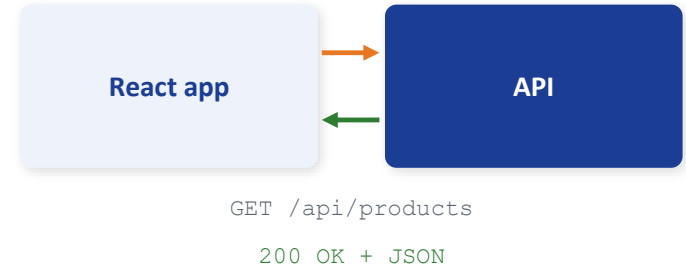
PART 4

REST API Architecture

- What an API is; what makes one RESTful
- Resources, endpoints and HTTP methods
- Status codes and the request/response lifecycle
- JSON and REST design best practices

What is an API? What is a Web API?

- **API = Application Programming Interface — a contract for software to talk to software**
 - You have used APIs all day: `Console.WriteLine` is the BCL's API; React hooks are React's API
- **A Web API exposes that contract over HTTP**
 - Client sends a request → server returns a response (usually JSON)
- **Why web APIs power full-stack systems:**
 - One backend serves web, mobile and partner integrations
 - Frontend and backend evolve and deploy independently
 - Clear security boundary (the only door to your data)



Restaurant analogy

The menu is the API contract; the waiter carries requests and responses (HTTP); the kitchen is your service + database. Diners never enter the kitchen — encapsulation at system scale.

REST — The Architectural Principles

Client–server

UI concerns separated from data/logic concerns; each evolves independently.

Stateless

Every request carries ALL context (incl. the JWT). No server session memory between calls.

Uniform interface

Resources identified by URLs; manipulated via standard HTTP methods; self-descriptive messages.

Resource-oriented

Model the domain as nouns (products, orders) — not actions (getProduct, doOrder).

Cacheable

Responses declare cacheability — GETs can be cached by browsers/CDNs for speed.

Layered system

Clients can't tell if they talk to the API, a gateway, or a load balancer — layers are invisible.

REST (Roy Fielding, 2000) is a STYLE, not a protocol or standard — 'RESTful' means following these constraints. Statelessness is the one you must never violate: it is what lets APIs scale horizontally.

Resources and Endpoint Design

Rule	✓ Good	✗ Avoid
Nouns, not verbs — the METHOD is the verb	GET /api/products	GET /api/getAllProducts
Plural collection names	/api/products, /api/orders	/api/product, /api/orderList
Identify items by id in the path	/api/products/42	/api/products?id=42 (for identity)
Nest ONLY true ownership, max 1 level	/api/orders/7/items	/api/customers/3/orders/7/items/2
Filters/paging via query string	/api/products?category=it&page=2	/api/products/it/page2
Consistent casing & style	kebab-or-lowercase, no trailing /	/api/ProductItems/, mixedStyles

A resource is a domain noun (usually an entity or aggregate from your ER diagram). The endpoint set for LankaMart almost writes itself from the Lecture 03 schema: /products, /categories, /orders, /orders/{id}/items, /auth/login.

HTTP Methods — The Verbs of REST

Method	Meaning	Example	Idempotent?	Body?
GET	Read a resource/collection — never changes state	GET /api/products/42	Yes (and safe)	No
POST	Create a new resource (server assigns id)	POST /api/orders	No	Yes
PUT	Replace a resource entirely	PUT /api/products/42	Yes	Yes
PATCH	Partially update specified fields	PATCH /api/products/42	Not guaranteed	Yes
DELETE	Remove a resource	DELETE /api/products/42	Yes	Usually no

Idempotent = repeating the request gives the same result. Why it matters: networks retry. Retrying DELETE /products/42 is harmless; retrying POST /orders can charge a customer twice — handle duplicates deliberately.

HTTP Status Codes — The Response Contract

2xx — Success

200 OK

GET/PUT/PATCH succeeded; body included

201 Created

POST succeeded; Location header points to the new resource

204 No Content

Success with nothing to return (DELETE)

4xx — Client's fault

400 Bad Request

Validation failed; malformed input

401 Unauthorized

Not authenticated (missing/invalid token)

403 Forbidden

Authenticated but not permitted

404 Not Found

Resource doesn't exist

409 Conflict

State conflict (duplicate email)

5xx — Server's fault

500 Internal Server Error

Unhandled exception — your bug

503 Service Unavailable

Overloaded / down for maintenance

Request → Response Lifecycle, and JSON

1. React fetch()

2. HTTP request

3. Middleware pipeline

4. Routing → Controller

5. Service → EF Core → PostgreSQL

6. JSON response

```
// HTTP request
GET /api/products/42 HTTP/1.1
Host: api.lankamart.lk
Authorization: Bearer eyJhbGc...
Accept: application/json
```

```
// HTTP response
HTTP/1.1 200 OK
Content-Type: application/json
```

```
{ "id": 42, "name": "Wireless Mouse",
  "price": 2500.00, "inStock": true }
```

JSON: lightweight, human-readable, language-neutral. ASP.NET Core serializes objects ↔ JSON automatically (System.Text.Json), converting PascalCase C# to camelCase JSON by default.

REST API Design Best Practices



Consistency above all

- Same naming style, casing, error shape and paging pattern on EVERY endpoint
- Consumers learn your API once



Paginate collections

- Never return unbounded lists:
`?page=1&pageSize=20`
- Return total count metadata for UIs



Version from day one

- `/api/v1/products` — breaking changes go to v2
- Old clients keep working



Uniform error shape

- Problem Details (RFC 9457) for all errors
- Machine-readable type + human title + traceId



Secure by default

- HTTPS only · [Authorize] unless explicitly public
- Validate every input (next part)



Document as you build

- Swagger/OpenAPI generated from code
- The docs ARE the contract — keep them honest

Checkpoint 03 – REST Design



Pairs — 5 minutes: design and defend

Q1

Design the endpoint (method + URL + success status) for: 'customer cancels order 17'. Two defensible answers exist — give one and justify it.

Q2

A teammate returns 200 OK with { "error": "product not found" }. Explain to them — kindly — everything wrong with this.

Q3

PUT /api/products/42 with body { "price": 2600 } wiped the product's name. What happened, and which method should have been used?

Q4

Why must POST /api/orders NOT be retried blindly on timeout, while DELETE /api/orders/17 can be?

PART 5

Building & Testing the API in ASP.NET Core

- MVC-style structure: controllers, services, models & DTOs
- Routing, attribute routing and endpoint mapping
- Request handling and JSON response formatting
- Testing with Swagger / OpenAPI and Postman

Clean ASP.NET Core Project Structure

folder layout

```
LankaMart.Api/  
├─ Program.cs           // startup, DI, middleware  
├─ appsettings.json     // configuration (no secrets!)  
├─ Controllers/  
│   └─ ProductsController.cs // HTTP layer  
├─ Services/  
│   ├── IProductService.cs // contract  
│   └─ ProductService.cs   // business logic  
├─ Models/  
│   └─ Product.cs          // domain entity  
├─ Dtos/  
│   ├── CreateProductDto.cs // input shape  
│   └─ ProductDto.cs        // output shape  
└─ Repositories/          // data access (next lecture)
```

Why this layout?

- Folders mirror responsibilities — find anything fast
- Each layer depends only on the one below via interfaces
- New feature = one file per folder (Orders → 5 files)
- Scales from assignment to enterprise unchanged
- Markers can navigate it instantly

Models and DTOs — and Why They Differ

```
// Models/Product.cs - internal domain entity
public class Product {
    public long Id { get; set; }
    public string Name { get; set; } = "";
    public decimal Price { get; set; }
    public int Stock { get; set; }
    public string SupplierCostNote = ""; // internal!
}
```

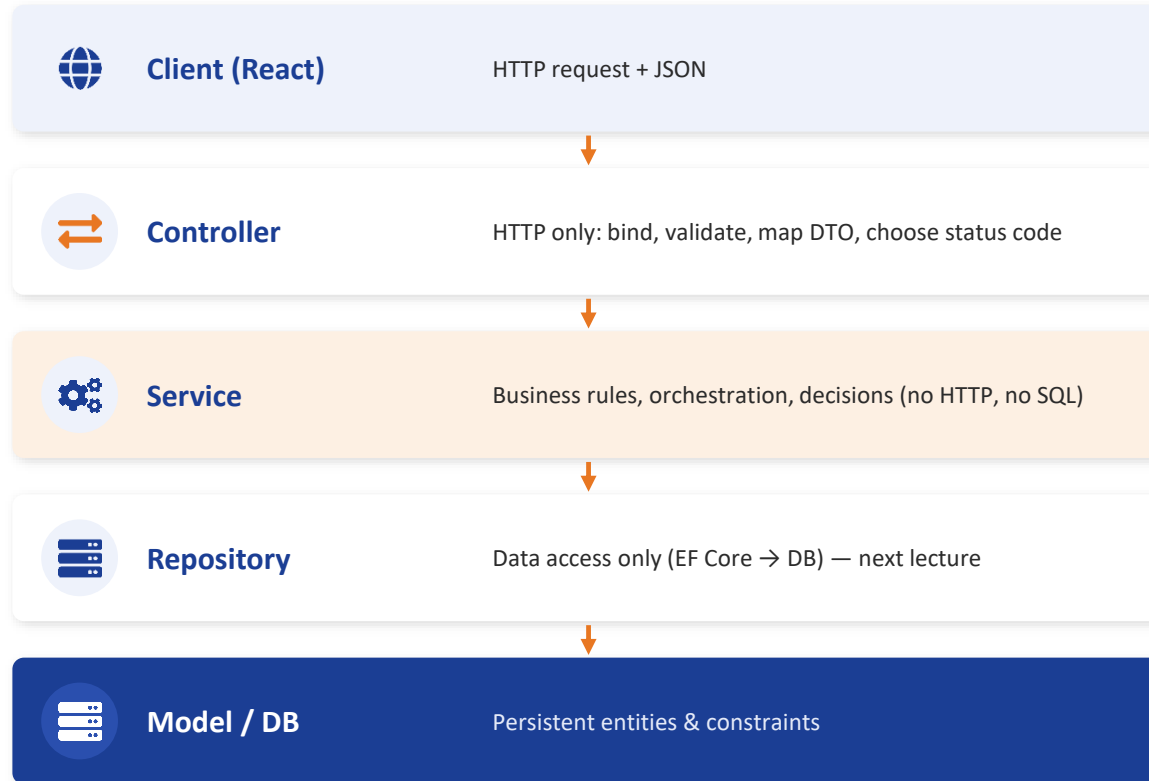
```
// Dtos - what crosses the API boundary
public record CreateProductDto(
    string Name, decimal Price, int Stock);

public record ProductDto(
    long Id, string Name, decimal Price,
    bool InStock); // no internal fields exposed
```

Model ≠ DTO

- **Model** = internal shape (maps to the DB)
- **DTO** = Data Transfer Object = the public contract
- Hide internal/sensitive fields from clients
- Input DTO ≠ output DTO (no Id on create)
- Decouples API from database changes
- record = concise, immutable DTO type

Controller → Service → Model Workflow



Separation of concerns

- Each box: one reason to change
- Controller thin, service rich
- Test the service without HTTP
- Swap the repo without touching controllers
- Same pattern as Lecture 03's diagram — now in C#

Controllers, Attribute Routing & Endpoint Mapping

ProductsController.cs

```
[ApiController]                // model binding + 400s
[Route("api/[controller]")]     // → api/products
public class ProductsController : ControllerBase
{
    private readonly IProductService _svc;
    public ProductsController(IProductService svc)
        => _svc = svc;           // injected by DI

    [HttpGet]                    // GET api/products
    public async Task<IEnumerable<ProductDto>> GetAll()
        => await _svc.GetAllAsync();

    [HttpGet("{id}")]           // GET api/products/42
    public async Task<ActionResult<ProductDto>> Get(long id)
    {
        var p = await _svc.GetByIdAsync(id);
        return p is null ? NotFound() : Ok(p);
    }
}
```

Reading it

- [ApiController] auto-returns 400 on invalid models
- [Route] + [Http*] attributes map URL+verb to a method
- [controller] token = class name minus 'Controller'
- Service arrives via constructor injection
- Ok(), NotFound(), Created() set the status code
- async Task<...> for non-blocking I/O

Services and Dependency Injection in Practice

```
// Services/IProductService.cs – the contract
public interface IProductService {
    Task<IEnumerable<ProductDto>> GetAllAsync();
    Task<ProductDto?> GetByIdAsync(long id);
    Task<ProductDto> CreateAsync(CreateProductDto dto);
}

// Services/ProductService.cs – the logic
public class ProductService : IProductService {
    public async Task<ProductDto> CreateAsync(
        CreateProductDto dto) {
        if (dto.Price < 0)
            throw new ValidationException("Price < 0");
        /* map, persist, return ProductDto */
    }
}
```

```
// Program.cs – register once
builder.Services
    .AddScoped<IProductService,
        ProductService>();
```

DI lifetimes

- **Scoped** — one per request (most services)
- **Transient** — new every time
- **Singleton** — one for the whole app
- **Depend on the INTERFACE**; swap implementations freely

Reading Route, Query and Body Parameters

```
// 1) Route parameter - part of the path
[HttpGet("{id}")]
public ... Get(long id) { ... }      // GET /products/42

// 2) Query parameters - filtering & paging
[HttpGet]
public ... Search(string? category,
                  int page = 1, int pageSize = 20)
// GET /products?category=it&page=2

// 3) Request body - JSON → DTO
[HttpPost]
public ... Create([FromBody] CreateProductDto dto)
// POST /products { "name": "Mouse", ... }
```

Binding sources

- **Route** → identity (which item)
- **Query** → options (filter, sort, page)
- **Body** → the payload (create/update data)
- ASP.NET Core binds by name & source automatically
- Defaults make query params optional
- Never read raw JSON by hand — bind to a DTO

Returning Proper Responses & JSON

```
[HttpPost]
public async Task<ActionResult<ProductDto>> Create(
    CreateProductDto dto) {
    var created = await _svc.CreateAsync(dto);
    return CreatedAtAction(nameof(Get),
        new { id = created.Id }, created); // 201 + Location
}

// helpers map intent → status code
Ok(dto) // 200 OK
CreatedAtAction // 201 Created (+ where to find it)
NoContent() // 204 (successful delete/update)
NotFound() // 404
BadRequest(err) // 400
```

JSON by default

ASP.NET Core serializes returned objects to JSON automatically (System.Text.Json). camelCase output by convention — matches JavaScript clients.

200 OK body

```
{
  "id": 42,
  "name": "Mouse",
  "price": 2500.00,
  "inStock": true
}
```

Match the status code to what happened: created → 201, found → 200, deleted → 204, missing → 404. Don't return 200 for everything.

API Testing with Swagger / OpenAPI

- **OpenAPI = a standard, machine-readable description of your API**
 - Swagger UI renders it as an interactive test page in the browser
- **Ships with the ASP.NET Core Web API template — almost zero setup**
- **Try endpoints live: fill parameters, Execute, see real responses**
- **Auto-documents routes, parameters, DTO shapes and status codes**
- **Frontend & mobile teams can generate client code from the spec**
 - Great first stop before reaching for Postman

Swagger UI — /swagger

GET

/api/products

GET

/api/products/{id}

POST

/api/products

PUT

/api/products/{id}

DELETE

/api/products/{id}

► Try it out → Execute → 200 OK

Testing with Postman — A Practical Workflow

1 Set up

Create a collection per API; an environment var baseUrl = https://localhost:5001

2 Send requests

Build GET/POST/PUT/DELETE with headers & JSON body; inspect status, body, time

3 Save & organise

Save each request; group by resource; share the collection with the team

4 Assert

Add test scripts: status is 200, body has an id, response < 500 ms

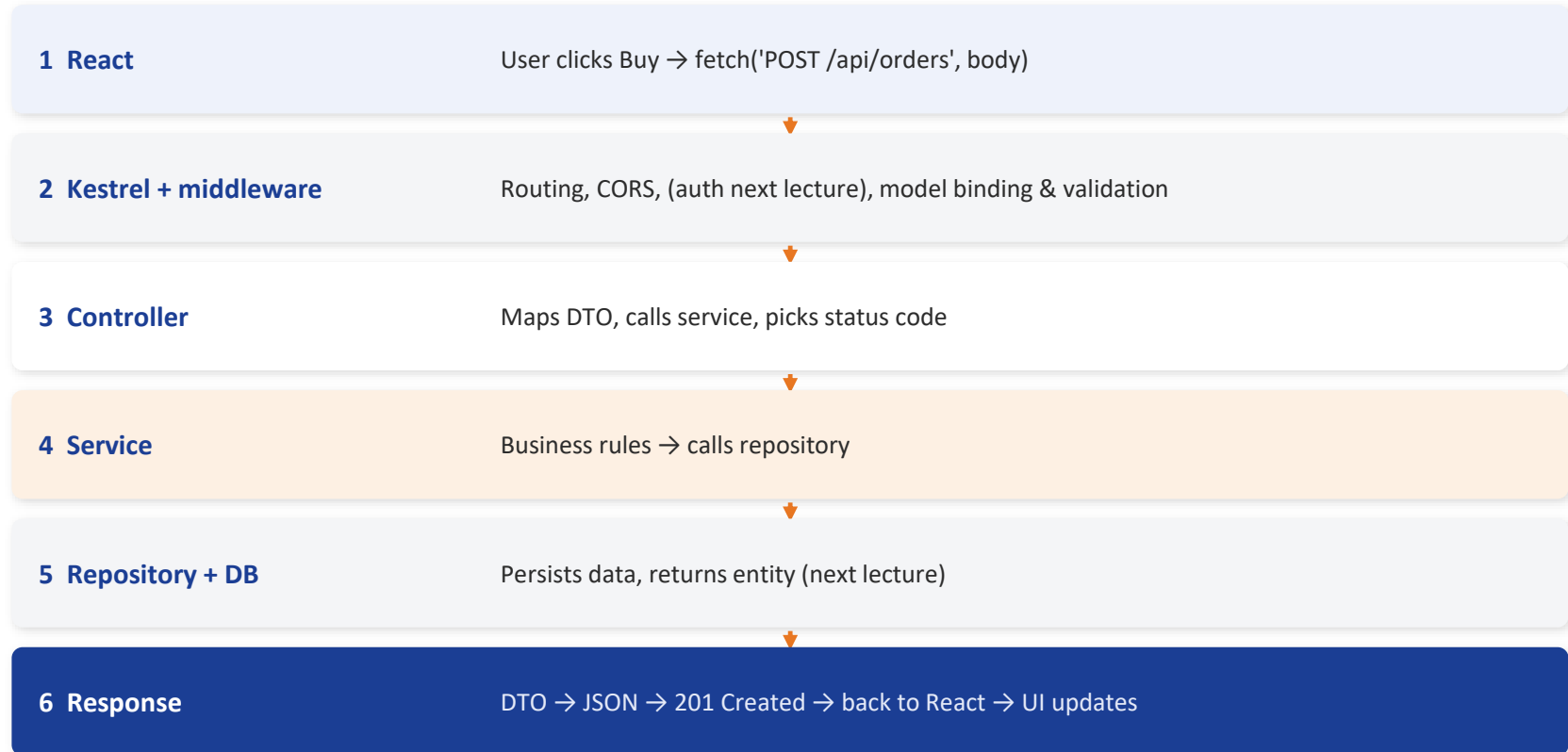
5 Automate

Run the whole collection (Runner / Newman) in CI on every push

Swagger vs Postman

- Swagger: quick manual checks, lives with the code, always in sync
- Postman: saved collections, scripted assertions, environments, automation
- **Use both: Swagger while coding, Postman for repeatable test suites**

End-to-End: From Browser Click to JSON Response



PART 6

Backend Best Practices: Maintainability, Scalability & Security

- Layered architecture & separation of concerns (recap in practice)
- Validation, exception handling & logging
- Secure configuration — no hardcoded secrets
- Writing code that survives the assignment AND the real world

Layered Architecture & DI Best Practices



Respect the layers

- Controller → Service → Repository, one way only
- No SQL in controllers, no HTTP in services
- Each layer testable in isolation



Depend on abstractions

- Inject interfaces, not concrete classes
- Register in Program.cs with the right lifetime
- Swap implementations without edits upstream



Small, focused units

- One class, one responsibility (SRP)
- Short methods with clear verb names
- Feature = one file per folder



Consistency wins

- Same naming, same folder layout everywhere
- Predictable code is maintainable code
- New teammates ramp up fast

Validation, Exception Handling & Logging

```
// 1) Declarative validation on the DTO
public record CreateProductDto(
    [Required, StringLength(120)] string Name,
    [Range(0, 1_000_000)] decimal Price,
    [Range(0, int.MaxValue)] int Stock);
// [ApiController] auto-returns 400 + details

// 2) Global exception middleware (Program.cs)
app.UseExceptionHandler(); // one place for errors

// 3) Structured logging via injected logger
_logger.LogInformation("Created product {Id}",
    created.Id);
```

Principles

- **Validate at the edge; fail fast with 400**
- Handle errors in ONE place, not try/catch everywhere
- Map exceptions → correct status codes
- Log full detail server-side; return safe messages
- **Never expose stack traces to clients**
- Use a `traceId` to link reports to logs

Secure Configuration & Basic API Security



Configuration & secrets

- **NEVER hardcode secrets or commit them to Git**
- Connection strings, API keys, JWT secrets → environment variables / user-secrets
- appsettings.json for non-secret config only
- Different values per environment (dev/test/prod)
- Production secrets from the platform (Azure Key Vault, etc.)
- Read via IConfiguration / IOption<T>



Baseline API security

- **HTTPS everywhere (TLS) — encrypt in transit**
- Validate & sanitize all input (prevents injection)
- Authentication + authorization (JWT + RBAC — next lecture)
- Least-privilege database account
- CORS allow-list — only your frontend origin
- Don't leak details in errors; rate-limit sensitive endpoints

Case Study: Designing the LankaMart Backend API



Requirements → API design decisions (with justification — LO4)

Requirement	Endpoint / design	Justification
Browse & search products	GET /api/products?category=&page=	Collection + query params for filtering/paging; cache-friendly
View one product	GET /api/products/{id} → 200 / 404	Identity in the path; correct status codes
Staff add a product	POST /api/products → 201 + Location	Server assigns id; CreatedAtAction; input via CreateProductDto
Update / partial edit	PUT vs PATCH /api/products/{id}	PUT full replace, PATCH partial — avoids accidental field wipe
Place an order	POST /api/orders (service + transaction)	Business logic in service layer; not a controller concern
Clean, layered code	Controllers · Services · DTOs · DI	Maintainable, testable, scalable — meets the rubric

Same method applies to your assignment domain: property management, student management, clinic booking, event ticketing ...

Group Activity: Design REST Endpoints (20 min)



Groups of 3–4 · whiteboard or paper · present in 60 seconds

Scenario: Campus Library System

Members borrow books; librarians manage the catalogue; admins manage members. A member can borrow up to 5 books at once.

- **Task 1 — List the resources (nouns) for the system**
- **Task 2 — Design endpoints (method + path) for browse, view one, add, update, delete a book**
- **Task 3 — Design the 'borrow a book' endpoint; which layer holds the 5-book rule?**
- **Task 4 — Pick the right status code for: created, not found, borrow-limit exceeded**

Marking lens (= the rubric)

- Nouns as resources, plural, no verbs in paths?
- Identity in the path, options in the query?
- Correct HTTP method per action?
- Business rule placed in the SERVICE, not the controller?
- Sensible status codes (201 / 404 / 400 or 409)?

Quick Quiz — 6 Questions

1

Which type stores money exactly in C#?

A) double B) float C) decimal

2

What does the [ApiController] attribute give you automatically?

A) a database B) 400 on invalid models C) HTTPS

3

A controller needs IProductService. How does it get one?

A) new ProductService() B) constructor injection (DI) C) a global variable

4

POST that creates a resource should return...

A) 200 OK B) 201 Created C) 204 No Content

5

Why return a DTO instead of the entity?

A) it's faster B) hide internal fields / decouple C) required by C#

6

Where should the JWT signing secret live?

A) appsettings.json in Git B) hardcoded C) environment / secret store

Lecture Summary — Key Takeaways



C# is your backend language

Strongly typed, OOP, modern (properties, var, async, LINQ); decimal for money.



REST is the contract

Resources + HTTP methods + status codes + JSON; design URLs around nouns.



OOP structures the code

Encapsulation, inheritance, polymorphism, abstraction & interfaces drive clean, testable design.



Build it in layers

Controllers (HTTP) → Services (logic) → Repositories (data); DTOs at the boundary.



.NET + ASP.NET Core run it

Cross-platform, fast, batteries-included: DI and middleware out of the box.



Do it well

Validate, handle errors centrally, log, and keep secrets out of code.

Learning Outcomes, Next Lecture & Preparation

LO	Covered today by...
LO2	Writing C# OOP classes and building working ASP.NET Core REST endpoints (controllers, services, models)
LO3	Layered architecture, dependency injection, validation, exception handling, correct HTTP semantics & JSON
LO4	Justifying C#/.NET, ASP.NET Core and REST design choices in the case study and endpoint-design activity



Next: Database, AuthN/AuthZ & Integration

Give the API memory and a lock: PostgreSQL schema design with EF Core, then JWT authentication and role-based access control securing these very endpoints.



Before the lab / next week

- Install the .NET 8 SDK; run `dotnet new webapi`
- Build & open the project; explore Swagger at `/swagger`
- Read: Microsoft Learn 'Create a web API with ASP.NET Core'
- Sketch the resource list for YOUR assignment domain

References & Further Reading

- **Official documentation (primary sources)**

- Microsoft Learn — C# documentation & language reference (learn.microsoft.com/dotnet/csharp)
- Microsoft Learn — ASP.NET Core fundamentals & 'Create a web API' tutorial
- Microsoft Learn — Dependency injection, configuration & logging in .NET
- OpenAPI Specification (openapis.org) · Swagger documentation (swagger.io)

- **Standards & guidelines**

- RFC 9110 — HTTP Semantics (methods & status codes) · RFC 8259 — JSON
- Microsoft REST API Guidelines · OWASP API Security Top 10 (owasp.org)

- **Recommended texts**

- Albahari, J. (2024) C# 12 in a Nutshell. O'Reilly Media.
- Price, M. J. (2023) C# 12 and .NET 8 – Modern Cross-Platform Development. Packt.
- Lock, A. (2023) ASP.NET Core in Action, 3rd edn. Manning Publications.